Project: **Spectral Clustering**

Presented by RAKOTOARIMANANA Joy Sandra Hasina

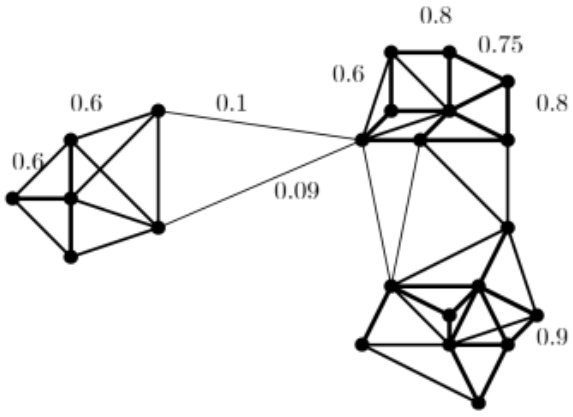
Introduction

This project explores mathematical foundations of clusterings. Spectral clustering is an unsupervised data partitioning method based on graph theory and linear algebra. Unlike classical methods such as k-means which operate directly on the coordinates of points in space, spectral clustering exploits the similarity structure between data by constructing a graph where each node represents a data point and each edge represents a similarity between two points.
For a set of data points $\{x_1, \ldots, x_n\}$ and a notion of similarity $s_{ij} \geq 0$, between all pairs of data points x_i and x_j, the goal of clustering is to partition (classify) the data points into several groups such that points in the same group are similar and points in different groups are dissimilar to each other.

We represent the data as a similarity graph $G = \{V, E\}$ where V represents the set of vertices and E the set of edges. Each vertex v_i in this graph represents the data point x_i and two vertices v_i and v_j are connected if the similarity s_{ij} between the corresponding data points is strictly positive or greater than a certain threshold. The edge represents the connection between two vertices and carries a weight s_{ij} which quantifies this connection.

Notion on Graphs

Let $G = \{V, E\}$ be an undirected graph with V the set of vertices $V = \{v_1, v_2, \ldots, v_n\}$. We assume that G is weighted, that is, each edge between two vertices v_i and v_j carries a weight $w_{ij} \geq 0$. The adjacency matrix for an undirected graph is the matrix $W = (w_{ij})_{i,j=1,\ldots,n}$. If $w_{ij} = 0$ then the two vertices are not connected by an edge. There exists edge between two vertices if and only if $w_{ij} > 0$. Since G is undirected, we have $w_{ij} = w_{ji}$, meaning that the adjacency matrix W is symmetric. Here is an example of graph of similarity



The algorithm starts by creating a similarity matrix from the data and then computes the Laplacian of this graph.

Steps to construct a similarity matrix:

1. We choose a similarity measure between two points x_i and x_j: Gaussian kernel function

$$s_{ij} = \exp\left(\frac{-||x_i - x_j||^2}{2\sigma^2}\right)$$

and $s_{ii} = 0$(this is a convention), where σ is a scale parameter whose value is fixed by the user

2. Decide the graph structure:
The objective is to model the local neighborhood relationships between data points.
   (a) The ε-neighborhood graph: we connect all points whose pairwise distances are less than ε
   (b) The k-nearest neighbor graphs: the goal is to connect the vertex v_i to the vertex v_j if v_j belongs to the k nearest neighbors of v_i. However, the neighborhood relation is not symmetric, therefore we transform the graph into an undirected one. We ignore the direction of edges and connect v_i and v_j if one of them is among the k nearest neighbors of the other.
   (c) The fully connected graph: here we simply connect all points that have positive similarity with each other and we weight all edges by s_{ij} .

3. We construct the similarity matrix: for a similarity function that models local neighborhood relations. We choose the Gaussian similarity function where the parameter σ controls the width of the neighborhoods.

Laplacian Matrix

We distinguish different types of graph Laplacians. We always assume that the graph G is undirected and weighted with adjacency matrix W where $w_{ij} = w_{ji} \geq 0$

The unnormalized Graph Laplacians

The unnormalized Laplacian is defined by:

$$L = D - W$$

where the degree matrix D is defined as the diagonal matrix with degrees $d_1, \ldots, d_n$ on the diagonal and we define the degree of a vertex v_i by

$$d_i = \sum_{j=1}^{n} w_{ij}$$

Proposition (Properties of L)

1. For each vector $f \in \mathbb{R}^n$, we have

$$f'Lf = \frac{1}{2}\sum_{i,j=1}^{n} w_{ij}(f_i - f_j)^2$$

2. L is symmetric and positive semi-definite
3. The smallest eigenvalue of L is 0, and the corresponding eigenvector is the constant vector 1.
4. L has n real and non-negative eigenvalues $0 = \lambda_1 \leq \lambda_2 \cdots \leq \lambda_n$

Proof :

1. We have
$$f'Lf = f'Df - f'Wf = \sum_{i=1}^{n} d_i f_i^2 - \sum_{i,j=1}^{n} f_i f_j w_{ij}$$
or
$$\frac{1}{2}.\sum_{i,j}^{n} w_{ij}(f_i - f_j)^2 = \frac{1}{2}.\left(\sum_{i,j} w_{ij} f_i^2 - 2\sum_{i,j} w_{ij} f_i f_j + \sum_{i,j} w_{ij} f_j^2\right)$$
since
$$d_i = \sum_{j=1}^{n} w_{ij} \text{ and } d_j = \sum_{i=1}^{n} w_{ij} = \sum_{i=1}^{n} w_{ji} \text{ then } d_i = d_j$$
$$= \frac{1}{2}\left(2\sum_{i=1}^{n} d_i f_i^2 - 2\sum_{i,j=1}^{n} f_i f_j w_{ij}\right) = f'Lf$$
where f' is the transpose of f.
2. Since W and D are symmetric: $w_{ij} = w_{ji}$, then L is symmetric. L is positive semi-definite because $f'Lf \geq 0$, which is a direct consequence of 1 for all $f \in \mathbb{R}^n$.
3. The smallest eigenvalue of L is 0, and the corresponding eigenvector is the constant vector 1 because $W1 = M$ with $M = (m_{i1})_{1 \leq i \leq n}$ a matrix with n rows and one column such that $m_{i1} = \sum_{j=1}^{n} w_{ij} = d_{ii}$ and $D1 = M$. Thus
$$L1 = M - M = 01 = 0$$
4. Since L is a matrix with n rows and n columns, it admits n eigenvalues (not necessarily distinct) which are positive according to 2.

Normalized Graph Laplacians

1. Symmetric normalized Laplacian
$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}$$
2. Random walk normalized Laplacian
$$L_{\text{rw}} = D^{-1} L = I - D^{-1} W$$

Proposition

1. For each $f \in \mathbb{R}^n$, we have
$$f' L_{sym} f = \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} \left(\frac{f_i}{\sqrt{d_i}} - \frac{f_j}{\sqrt{d_j}} \right)^2$$
2. λ is an eigenvalue of L_{rw} with u the eigenvector associated with λ if and only if λ is an eigenvalue of L_{sym} with associated eigenvector $\omega = D^{\frac{1}{2}} u$
3. λ is an eigenvalue of L_{rw} with u the associated eigenvector if and only if λ and u solve the generalized problem $Lu = \lambda Du$
4. 0 is the smallest eigenvalue of L_{sym} and L_{rw}, with associated eigenvector $D^{1/2}1$ for L_{sym} and 1 for L_{rw}.
5. L_{sym} and L_{rw} have n non-negative real eigenvalues
$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$$

Proof:

1. Let $f \in \mathbb{R}^n$. We have $D^{-1/2} = diag\left(\frac{1}{\sqrt{d1}}, \cdots, \frac{1}{\sqrt{d_n}} \right)$ and W is a symmetric matrix, then $D^{-1/2} W D^{-1/2} = \left(\frac{w_{ij}}{\sqrt{d_i d_j}} \right)_{1 \leq i,j \leq n}$. We have then
$$f' L f = f' I f - f' D^{-1/2} W D^{-1/2} f$$
With $f' I f = \sum_{i=1}^{n} f_i^2$ and
$$f' D^{-1/2} W D^{-1/2} f = \sum_{i=1}^{n} \sum_{j=1}^{n} \frac{f_i f_j w_{ij}}{\sqrt{d_i d_j}}$$
Then
$$f' L f = \sum_{i=1}^{n} f_i^2 - \sum_{i=1}^{n} \sum_{j=1}^{n} \frac{f_i f_j w_{ij}}{\sqrt{d_i d_j}}$$
Let us expand the expression $\frac{1}{2} \sum_{i,j=1}^{n} w_{ij} \left(\frac{f_i}{\sqrt{d_i}} - \frac{f_j}{\sqrt{d_j}} \right)^2$:
$$\left(\frac{f_i}{\sqrt{d_i}} - \frac{f_j}{\sqrt{d_j}} \right)^2 = \frac{f_i^2}{d_i} - 2 \frac{f_i f_j}{\sqrt{d_i d_j}} + \frac{f_j^2}{d_j}$$
then
$$\sum_{j=1}^{n} w_{ij} \left(\frac{f_i}{\sqrt{d_i}} - \frac{f_j}{\sqrt{d_j}} \right)^2 = f_i^2 - 2 \sum_{j=1}^{n} w_{ij} \frac{f_i f_j}{\sqrt{d_i d_j}} + \sum_{j=1}^{n} w_{ij} \frac{f_j^2}{d_j}$$

$$\frac{1}{2}\sum_{i,j=1}^{n} w_{ij}\left(\frac{f_i}{\sqrt{d_i}}-\frac{f_j}{\sqrt{d_j}}\right)^2=\frac{1}{2}\sum_{i=1}^{n} f_i^2-\sum_{i,j=1}^{n} w_{ij}\frac{f_i f_j}{\sqrt{d_i d_j}}+\frac{1}{2}\sum_{j=1}^{n} f_j^2$$

and

$$\frac{1}{2}\sum_{i,j=1}^{n} w_{ij}\left(\frac{f_i}{\sqrt{d_i}}-\frac{f_j}{\sqrt{d_j}}\right)^2=\sum_{i=1}^{n} f_i^2-\sum_{i,j=1}^{n} w_{ij}\frac{f_i f_j}{\sqrt{d_i d_j}}$$

since

$$\sum_{i=1}^{n} f_i^2=\sum_{j=1}^{n} f_j^2$$

hence the result.

2. Let λ be an eigenvalue of L_{rw} and u the associated eigenvector. Then we have $L_{rw}u=\lambda u$, that is $D^{-1}Lu=\lambda u$. By multiplying by $D^{1/2}$, we obtain $D^{-1/2}LD^{-1/2}D^{1/2}u=\lambda D^{1/2}u$.
   We know that $L_{sym}=D^{-1/2}LD^{-1/2}$, thus

$$L_{sym}\left(D^{1/2}u\right)=\lambda\left(D^{1/2}u\right)$$

   By setting $w=D^{1/2}u$, we obtain that λ is an eigenvalue of L_{sym} with w the associated eigenvector. Since we used an equality, the proof works vice versa and we obtain the equivalence.

3. Suppose that λ and u solve the problem $Lu=\lambda Du$. Multiply the problem by D^{-1} on the left. Then we obtain

$$D^{-1}Lu=D^{-1}\lambda Du=\lambda D^{-1}Du=\lambda u$$

   Since $L_{rw}=D^{-1}L$, we obtain

$$L_{rw}u=\lambda u$$

   that is, u is the eigenvector associated with the eigenvalue λ of L_{rw}. Since the proof uses an equality, we indeed obtain the equivalence.

4. From 1), all eigenvalues of L_{sym} are positive, and from 2) all eigenvalues of L_{rw} are also positive. Let us now show that 0 is an eigenvalue of L_{rw} with associated eigenvector the constant vector 1.
   We have $L_{rw}=I-D^{-1}W$ and $D^{-1}W=A=(a_{ij})_{i,j=1,\cdots,n}$ such that $a_{ij}=\dfrac{w_{ij}}{d_{ii}}$
   Thus $L_{rw}1=1-A1$. We also have $A1=B$ with $B=(b_{i1})_{i=1,\ldots,n}$ is a column vector (matrix with n rows and one column) such that

$$b_{i1}=\sum_{j=1}^{n}\frac{w_{ij}}{d_{ii}}=\frac{1}{d_{ii}}\sum_{j=1}^{n} w_{ij}$$

Since $d_{ii} = \sum_{j=1}^{n} w_{ij}$, we obtain $b_{i1} = 1$. Hence $B = 1$

Therefore

$$L_{rw}1 = 1 - 1 = 0$$

Thus 0 is an eigenvalue of L_{rw} with associated eigenvector 1, and from 2), 0 is an eigenvalue of L_{sym} with associated eigenvector $D^{1/2}1$. Hence we obtain the result.

5. From 1, we have $f'L_{sym}f \geq 0$. D and W are symmetric, therefore L_{sym} is also symmetric and positive semi-definite (i.e., its eigenvalues are non-negative). From 2, L_{sym} and L_{rw} have the same number of eigenvalues, which is equal to n. Since L_{sym} and L_{rw} are real matrices, their eigenvalues are also real.

Principle of the Spectral Clustering Algorithm

This principle follows the following steps:

1. Construction of the similarity graph: we create the similarity matrix or adjacency matrix from the data. To do this, we choose a similarity measure which is a Gaussian kernel function: $s_{ij} = \exp\left(\frac{-||x_i - x_j||^2}{2\sigma^2}\right)$
2. Computation of the graph Laplacian
3. Spectral decomposition: We extract the first k eigenvectors associated with the k smallest eigenvalues of the Laplacian. These first k eigenvectors are used to project the data into a lower-dimensional space
4. Projection into the spectral space: the data are then projected into a new space defined by these eigenvectors where the clusters become more easily separable
5. Final clustering: We apply the k-means algorithm in this transformed space to obtain the final clusters

Algorithms

Spectral Clustering with Unnormalized Laplacian

Algorithm 1 Spectral Clustering with Unnormalized Laplacian

Construct the adjacency matrix $W = (w_{ij})_{i,j=1,\dots,n}$
Compute the degree matrix $D = (d_{ij})_{i,j=1,\dots,n}$ which is a diagonal matrix where $d_{ii} = \sum_{j=1}^{n} w_{ij}$ and $d_{ij} = 0$ if $i \neq j$
Compute the unnormalized Laplacian $L = D - W$
Compute the first k eigenvectors $u_1, \dots, u_k$ of L
Let $U \in \mathbb{R}^{n \times k}$ be the matrix containing these k eigenvectors as columns
for $i = 1, \dots, n$ **do**
    Let $y_i \in \mathbb{R}^k$ be the row vector corresponding to the i-th row of U
end for
Cluster the points $(y_i)_{i=1,\dots,n}$ in $\mathbb{R}^k$ using the k-means algorithm into clusters $C_1, \dots, C_k$

Spectral Clustering with Random Walk Laplacian

Algorithm 2 Spectral Clustering with Random Walk Laplacian

1: Construct the adjacency matrix W of the weighted graph and the degree matrix D
2: Compute the random walk Laplacian

$$L_{rw} = I - D^{-1}W$$

3: Compute the first k eigenvectors $u_1, \ldots, u_k$ of the generalized eigenvalue problem

$$Lu = \lambda Du$$

4: Let $U \in \mathbb{R}^{n \times k}$ be the matrix containing the vectors $u_1, \ldots, u_k$ as columns
5: **for** $i = 1, \ldots, n$ **do**
6: Let $y_i \in \mathbb{R}^k$ be the row vector corresponding to the i-th row of U
7: **end for**
8: Apply the k-means algorithm to cluster the points $(y_i)_{i=1,\ldots,n}$ into clusters $C_1, \ldots, C_k$

Spectral Clustering with Symmetric Normalized Laplacian

Algorithm 3 Spectral Clustering with Symmetric Normalized Laplacian

1: Construct the adjacency matrix W and the degree matrix D
2: Compute the symmetric normalized Laplacian

$$L_{sym} = I - D^{-1/2}WD^{-1/2}$$

3: Compute the first k eigenvectors $u_1, \ldots, u_k$ associated with the k smallest eigenvalues of L_{sym}
4: Let $U \in \mathbb{R}^{n \times k}$ be the matrix containing the vectors $u_1, \ldots, u_k$ as columns
5: Form the matrix $T \in \mathbb{R}^{n \times k}$ from U by normalizing each row to have unit norm

$$t_{ij} = \frac{u_{ij}}{\left(\sum_k u_{ik}^2\right)^{1/2}}$$

6: **for** $i = 1, \ldots, n$ **do**
7: Let $y_i \in \mathbb{R}^k$ be the row vector corresponding to the i-th row of T
8: **end for**
9: Cluster the points $(y_i)_{i=1,\ldots,n}$ using the k-means algorithm into clusters $C_1, \ldots, C_k$

Now, we will apply k-means clustering and then use PCA to visualize the data in two dimensions. Before that, let's see how these two methods work

K-means algorithm

Definition

K-means is an unsupervised learning algorithm used to solve clustering problems.It follows a simple procedure consisting of classifying a set of data into a number of clusters k, which is set in advance. K-Means Clustering groups similar data points into clusters without needing labeled data

Principle of K-means algorithm

The algorithm will categorize the items into "k" groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. Initialization: We begin by randomly selecting k cluster centers or centroids
2. Assign each data point to the nearest centroid, forming clusters
3. Update: we recalculate the centroid of each cluster by averaging the points in this class
4. Repeat: This process repeats until the centroids no longer change or the maximum number of iterations is reached.

PCA algorithm

Definition

PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information.It helps us to remove redundancy, improve computational efficiency and make data easier to visualize and analyze.

Principle of Principal Component Analysis

PCA uses linear algebra to transform data into new features called principal components. The algorithm works as follows:

1. Standarize the data
2. Calculate the covariance matrix to see how features relate to each other if they increase or decrease together
3. It calculates eigenvectors (directions) and eigenvalues (importance) from the covariance matrix. PCA identifies new axes: the first component PC1 that is the direction of variance(most spread) and the second component PC2 that is the next best direction, perpendicular to PC1, and so on.
4. Select the top n components that capture most of the variance. Transform the original dataset by projecting it in these top components.This means we reduce the number of features (dimensions) .

Implementation

GitHub repository : `https://github.com/J-Sandra/Spectral_Clustering_project/blob/main/Spectral_clustering_project.ipynb`

Python Implementation

```python
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
import numpy as np
from sklearn.metrics.pairwise import euclidean_distances
from sklearn.neighbors import kneighbors_graph
from sklearn.cluster import KMeans
from scipy.linalg import eigh

def adjacency_matrix(X, sigma=1.0, k=5, mode='full'):
  """
Builds the adjacency matrix W from a dataset X.

Parameters:
-----------
X : ndarray (n_samples, n_features)
    Input data.
sigma : float
    Gaussian kernel parameter (kernel width).
k : int
    Number of neighbors for the k-NN graph.
mode : str
    'knn' for k-nearest neighbors, 'full' for complete graph.

Returns:
--------
W : ndarray (n_samples, n_samples)
    Weighted adjacency matrix.
   """

  n = X.shape[0]

  if mode == 'knn':

    # k-nearest neighbors graph (non-symmetric at first)
        A = kneighbors_graph(X, k, mode='connectivity', include_self=False
   )
        A = np.maximum(A.toarray(), A.toarray().T)  # Make symmetric
  elif mode == 'full':
     # Complete graph (all points connected)
        A = np.ones((n, n)) - np.eye(n)
  else:
        raise ValueError("Mode must be 'knn' or 'full'")

    # Computation of Euclidean distances
  dist = euclidean_distances(X, X)

    # Weights with Gaussian kernel
  W = np.exp(-dist**2 / (2 * sigma**2)) * A

  return W


```

```
def degree_matrix_from_W(W, return_vector=False):
  """
Computes the degree matrix D from a similarity matrix W.

Parameters
----------
W : array-like, shape (n, n)
    Similarity matrix (weighted). Must be square.
return_vector : bool, default=False
    If True, also returns the degree vector (shape (n,)).

Returns
-------
D : ndarray, shape (n, n)
    Diagonal degree matrix (D[i,i] = sum_j W[i,j]).
(degrees) : ndarray, shape (n,)
    (optional) degree vector.

"""
  W = np.asarray(W)
  if W.ndim != 2 or W.shape[0] != W.shape[1]:
      raise ValueError("W must be a square matrix (n x n).")

    # sum by row -> degree vector
  degrees = np.sum(W, axis=1)

    # build the diagonal matrix
  D = np.diag(degrees)

  if return_vector:
      return D, degrees
  return D

def laplacian_sym(D,W):#D^(-1/2)LD^(-1/2)
    L=D-W # unnormalized matrix
    if D.ndim == 2:
        d = np.diag(D)
    n = len(d)
    D_inv_sqrt = np.zeros((n, n))

    for i in range(n):
        if d[i] > 0:
            D_inv_sqrt[i, i] = 1 / np.sqrt(d[i])
        else:
            D_inv_sqrt[i, i] = 0.0  # to avoid division by zero
    L_sym = D_inv_sqrt @ L @ D_inv_sqrt
    return L_sym

def extract_k(L_sym,n):
   # Compute all eigenvalues and eigenvectors
    eigvals, eigvecs = eigh(L_sym)
    # 3. Determine k via the largest "gap" between eigenvalues
    # We only consider the first max_clusters eigenvalues
    # max_clusters = min(20, len(eigvals) - 1)
```

```
    # gaps = np.diff(eigvals[:max_clusters + 1])
    # k = np.argmax(gaps) + 1

    gaps = np.diff(eigvals[:n+1])
    k = np.argmax(gaps) + 1  # +1 because diff shifts the index by 1
    # 4. Take the corresponding k eigenvectors
    U = eigvecs[:, :k]
    # U is a numpy matrix of size (n, k)
    U_norm = U / np.linalg.norm(U, axis=1, keepdims=True)
    return U,k,U_norm

def distance(u,v):
    v=np.array(u)-np.array(v)
    r=np.sum(v*v)
    return np.sqrt(r)


def spectral_clustering(X, sigma=0.5, k_neighbors=5, mode='full',
    k_clusters=None, tol=1e-6):
    """
      Performs spectral partitioning on the data X.
      labels : ndarray (n_samples,)
      Cluster labels assigned to each point.
    """
    # 1. Build the adjacency matrix
    W = adjacency_matrix(X, sigma=sigma, k=k_neighbors, mode=mode)
    n = W.shape[0]

    # 2. Compute the degree matrix
    D, degrees = degree_matrix_from_W(W, return_vector=True)

    # 3. Compute the symmetric normalized Laplacian matrix
    L_sym = laplacian_sym(D, W)

    # 4. Extract the eigenvectors
    # If k_clusters is not specified, extract_k will try to determine it.
    if k_clusters is None:
        U, k_actual, U_norm = extract_k(L_sym, n)
        print(f"Number of clusters determined automatically : {k_actual}")
        k_to_use = k_actual
    else:
        # If k_clusters is specified, we take the first k_clusters
    eigenvectors
        eigvals, eigvecs = eigh(L_sym)
        U = eigvecs[:, :k_clusters]
        U_norm = U / np.linalg.norm(U, axis=1, keepdims=True)
        k_to_use = k_clusters
        print(f"Use of specified k_clusters: {k_to_use}")


    kmeans = KMeans(
    n_clusters=k_to_use,
    tol=tol,
    n_init=10,
```

```
    random_state=0
    )

    labels = kmeans.fit_predict(U_norm)

    return labels

def visualize_clustering(X, labels):
    plt.figure(figsize=(6,5))
    plt.scatter(X[:, 0], X[:, 1], c=labels)
    plt.title("Spectral Clustering")
    plt.show()

from sklearn.datasets import make_s_curve
X, y = make_s_curve(
  n_samples=500,
  noise=0.05,
 random_state=10
)
labels= spectral_clustering(X)
from sklearn.decomposition import PCA
X_2d = PCA(n_components=2).fit_transform(X)
visualize_clustering(X_2d, labels)
```

Experiment

We test the algorithm on synthetic datasets.

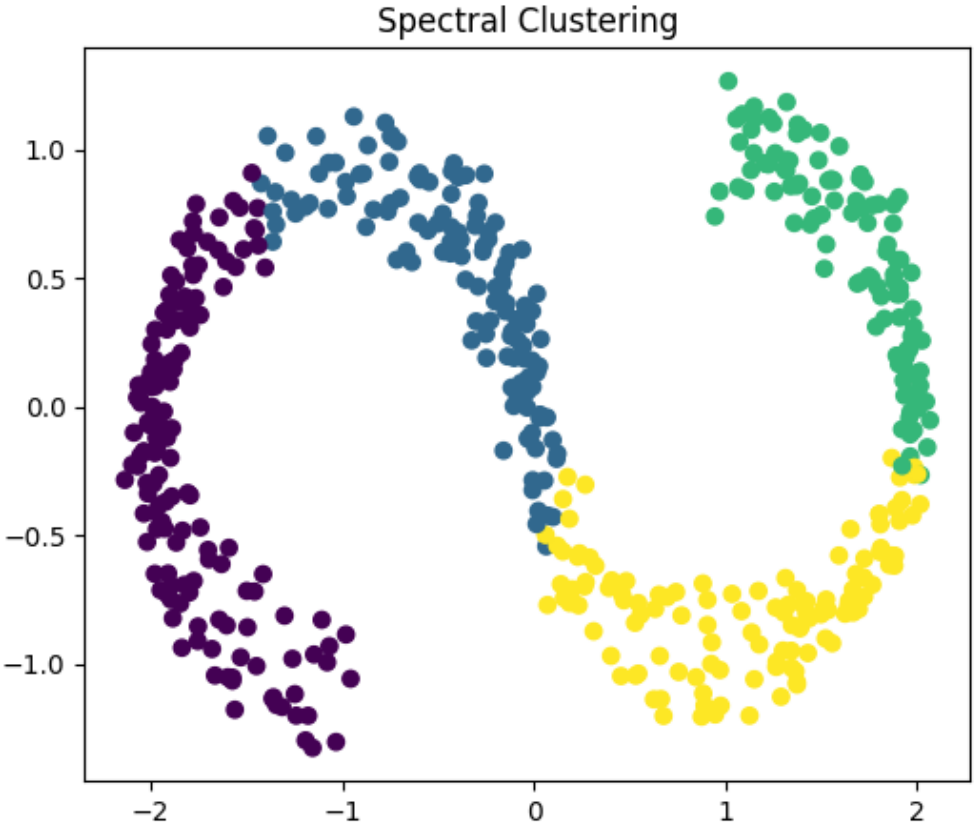


Figure 1: Spectral Clustering Result